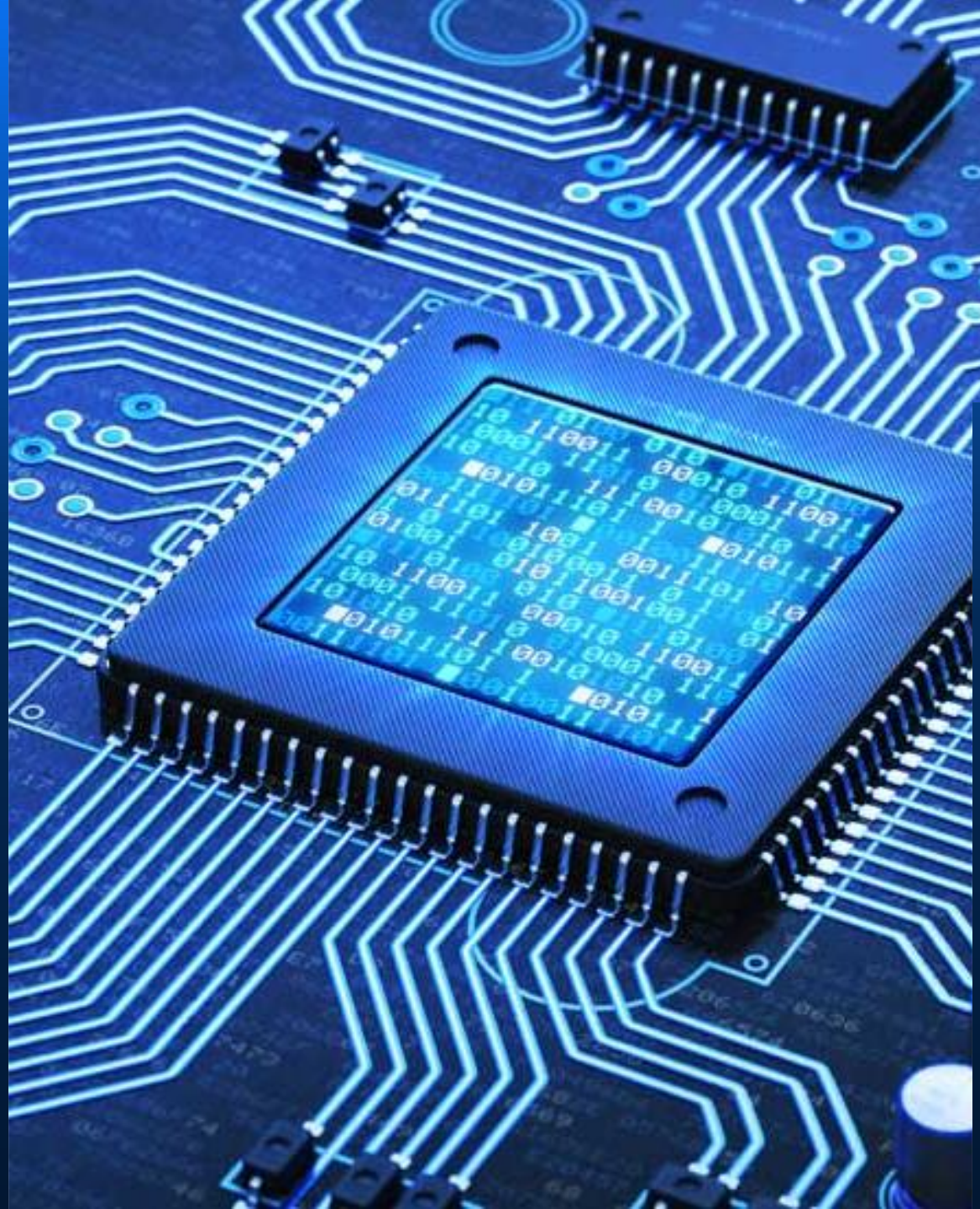


Computer Architecture

Practice Session 5

By: Eng. Abdulkareem Mohammed

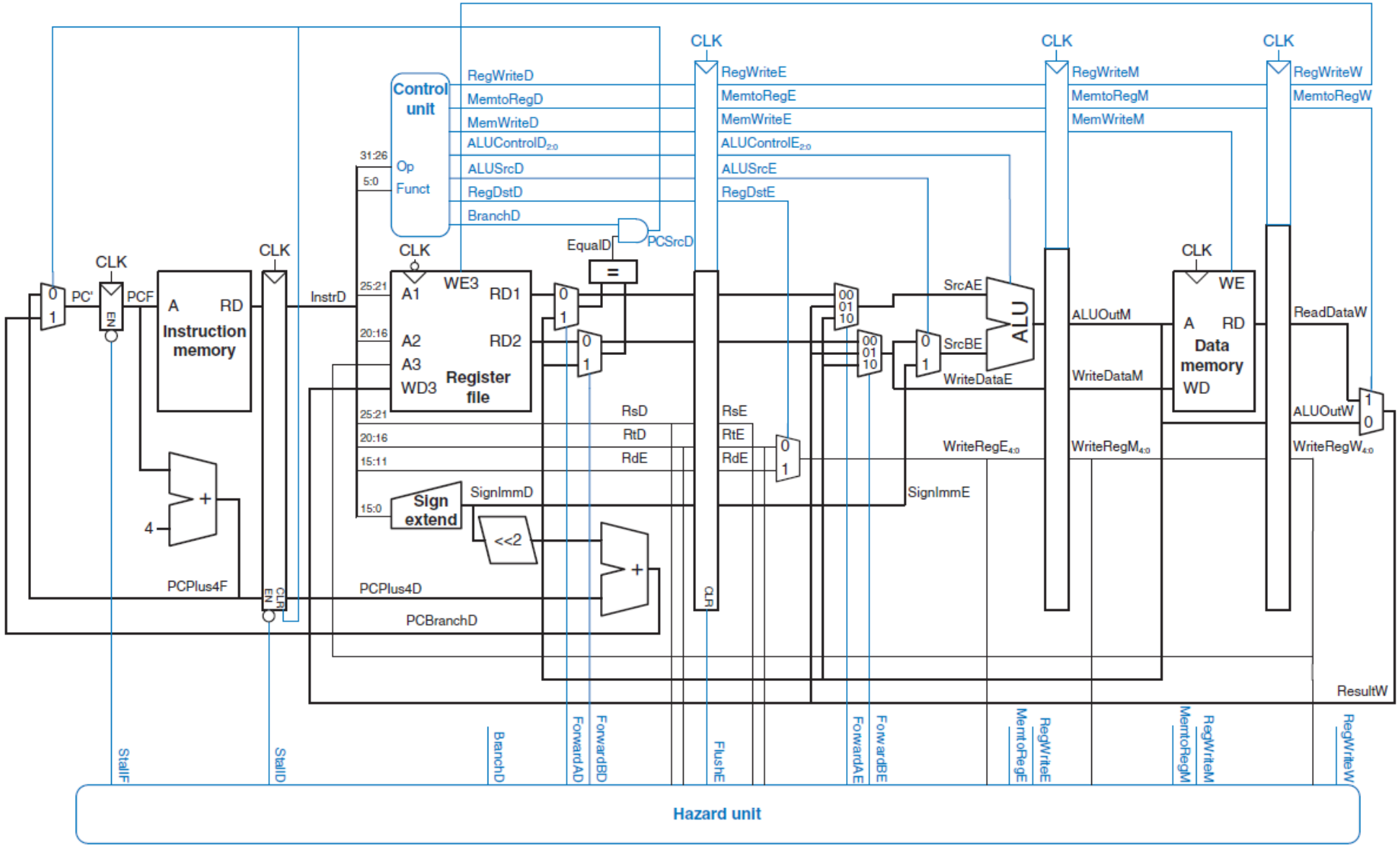
Senior HW Digital Design Engineer

Agenda

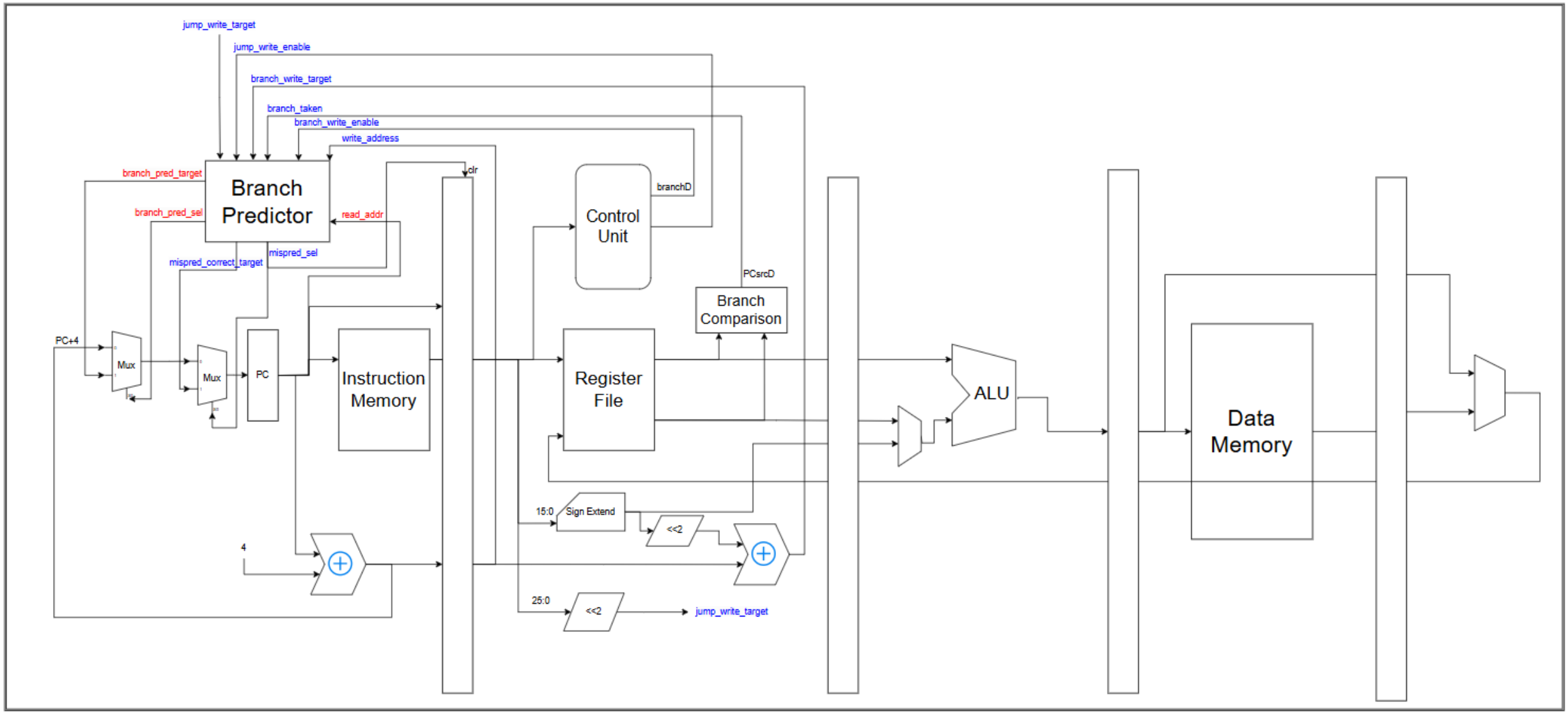
1. MIPS with Branch Prediction
2. Proposed Branch Predictor
3. MIPS Hierarchy
4. Testbench
5. Synthesis

1. MIPS with Branch Prediction

Pipelined MIPS Architecture



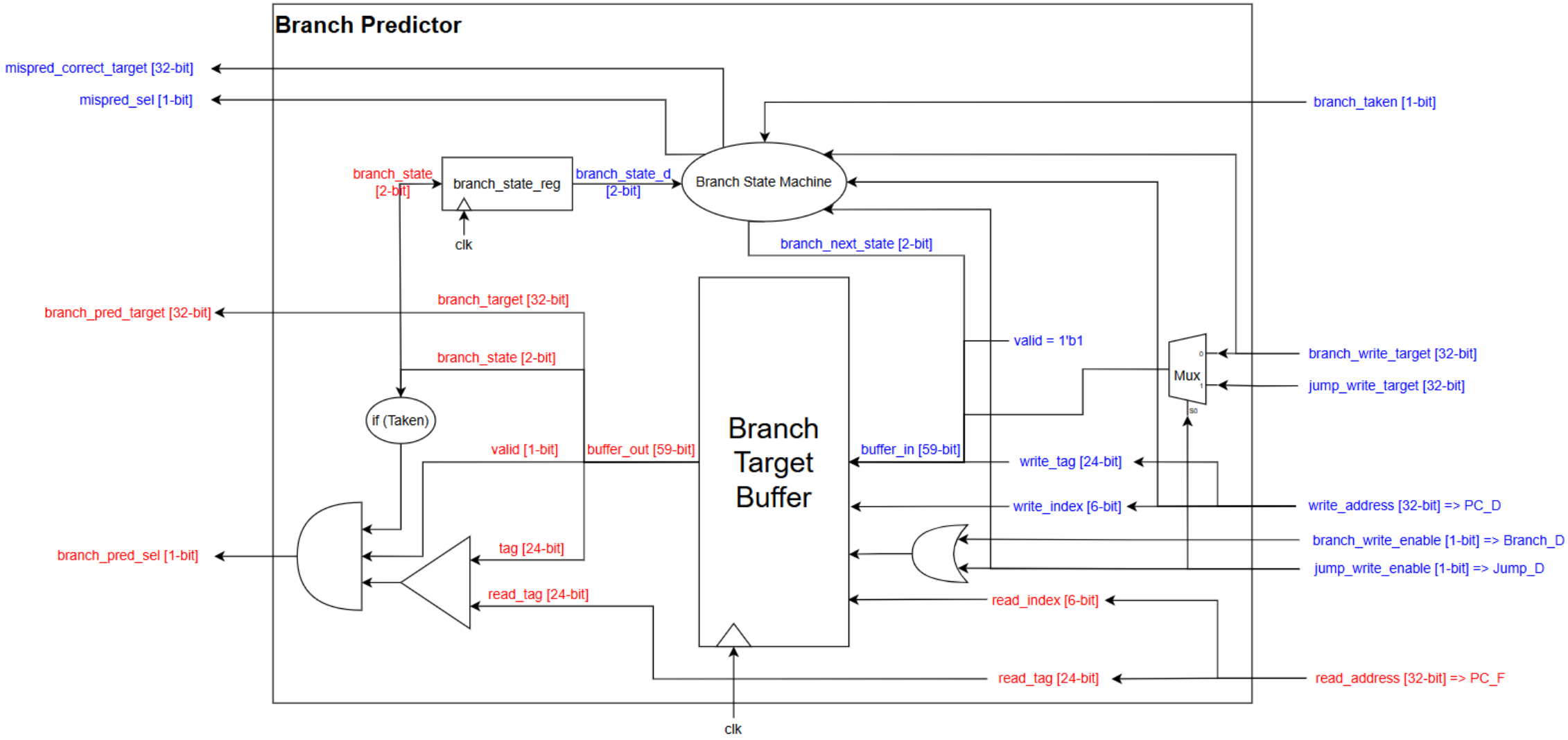
Proposed MIPS with Branch Prediction



2. Proposed Branch Predictor

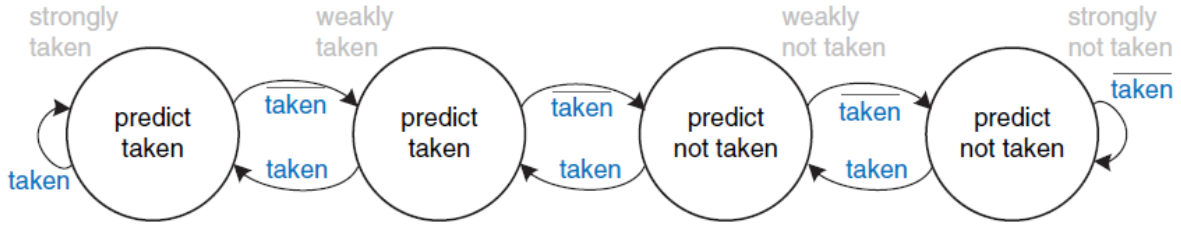
life.augmented

2. Proposed Branch Predictor



2. Proposed Branch Predictor

- Branch State Machine:



Branch state			branch_ taken	jump	Branch next state			mispred	mispred Correct target
Strongly not taken	0	0	0	0	Strongly not taken	0	0	0	0
	0	0	1	0	weakly not taken	0	1	1	branch_target
weakly not taken	0	1	0	0	Strongly not taken	0	0	0	0
	0	1	1	0	weakly taken	1	0	1	branch_target
weakly taken	1	0	0	0	weakly not taken	0	1	1	PC_D
	1	0	1	0	Strongly taken	1	1	0	0
Strongly taken	1	1	0	0	weakly taken	1	0	1	PC_D
	1	1	1	0	Strongly taken	1	1	0	0
x	x	x	x	1	Strongly taken	1	1	0	0

3. MIPS Hierarchy

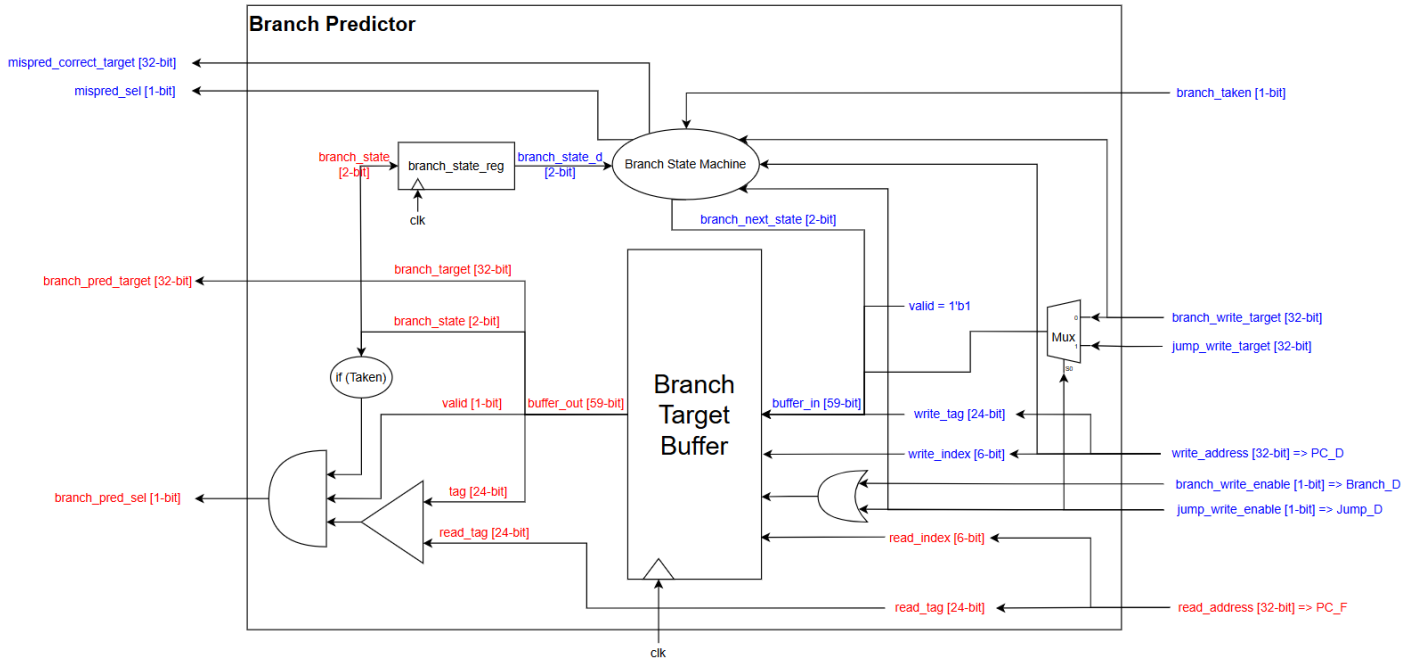
3. MIPS Hierarchy

- mips_tbench:
  - mips_top_inst
    - branch_predictor
    - fetch_stage
      - instruction_mem_inst
      - pc_reg
      - f2d_regs
    - decode_stage
      - reg_file_inst
      - branch_calc_inst
      - sign_extend_inst
      - ctrl_unit_inst
      - d2e_regs
    - excute_stage
      - alu_inst
      - e2m_regs
    - mem_stage
      - mem_data_mem_inst
      - m2w_regs
    - hazard_unit

4. Component Implementation

4. Component Implementation

```
module branch_predictor (
    input clk, rst_n,
    // For Fetch Stage
    input [31:0] read_address,
    output reg branch_pred_sel,
    output reg [31:0] branch_pred_target,
    // For Decode Stage
    input [31:0] write_address,
    input branch_write_enable,
    input jump_write_enable,
    input [31:0] branch_write_target,
    input [31:0] jump_write_target,
    input branch_taken,
    output reg mispred_sel,
    output reg [31:0] mispred_correct_target
);
```



5. Test & Simulation

life.augmented

5. Test & Simulation

Direct Testcase (7):

```
int c;
int main () {
    int i, j;
    for (i=0; i<1000; i++) {
        for (j=0; j<4; j++) {
            c++;
        }
    }
    return c;
}
```

```
addi $t0, $0, 0x03E8          ; $t0=1000
addi $t1, $0, 0x0004
addi $s0, $0, 0x0000
Outer_Loop:
            beq $s0, $t0, exit
            addi $s1, $0, 0x0000
            Inner_Loop:
                        beq $s1, $t1, exit_in
                        addi $s2, $s2, 0x0001
                        addi $s1, $s1, 0x0001
                        j Inner_Loop
            exit_in:
            addi $s0, $s0, 0x0001
            j Outer_Loop
exit:
addi $t2, $0, 0x0FA0
beq $s2, $t2, DONE
addiu $s0, $0, 0XDEAD
j SKIP
DONE:  addiu $s0, $0, 0XD08E
SKIP:  jr $ra
```

5. Test & Simulation

The inner for loop:

- 1st iteration($s1=0): 2 cycles (for beq), 2 cycles (for jump), as they are invalid in BTB, and 1 cycle for each addi, consuming 6 cycles.
- Within the loop ($s1=1,2,3): 1 cycle for each instruction due to the correct prediction, consuming 4 cycles.
- Last iteration ($s1=4): 2 cycles for beq and the next addi instruction will be flushed (misprediction).
- Note that: the inner 1st iteration for the outer 1st iteration will consume 6 cycles, but the inner 1st iteration for the remaining outer iteration will consume 4 cycles like the iterations within the loop.

#	1st Iteration	Within Loop	Last iteration	Total Calc	Total
Inner Loop	6	4	2	6+4*3+2	20
Inner Loop within the outer Loop	4	4	2	4+4*3+2	18

```
addi $t0, $0, 0x03E8
addi $t1, $0, 0x0004
addi $s0, $0, 0x0000
Outer_Loop:
        beq $s0, $t0, exit
        addi $s1, $0, 0x0000
        Inner_Loop:
                beq $s1, $t1, exit_in
                addi $s2, $s2, 0x0001
                addi $s1, $s1, 0x0001
                j Inner_Loop
        exit_in:
        addi $s0, $s0, 0x0001
        j Outer_Loop
exit:
addi $t2, $0, 0x0FA0
beq $s2, $t2, DONE
addiu $s0, $0, 0XDEAD
j SKIP
DONE:  addiu $s0, $0, 0XD08E
SKIP:  jr $ra
```

5. Test & Simulation

The outer for loop:

- 1st iteration($s0=0): 2 cycles (for beq), 2 cycles (for jump), as they are invalid in BTB, and 1 cycle for each addi, in addition to the consuming cycles for the inner for loop.
  #cycles = 2+2+1+1+(20) = 26 cycles
- Within the loop ($s1=1…999): 1 cycle for each instruction due to the correct prediction.
  #cycles = 1+1+1+1+(18) = 22 cycles
- Last iteration ($s1=4): 2 cycles for beq and the next addi instruction will be flushed (misprediction).

#	1st Iteration	Within Loop	Last iteration	Total Calc	Total
Inner Loop	6(4)	4	2	6+4*3+2	20(18)
Outer Loop	26	22	2	26+22*999+2	22,006

```
addi $t0, $0, 0x03E8
addi $t1, $0, 0x0004
addi $s0, $0, 0x0000
Outer_Loop:
        beq $s0, $t0, exit
        addi $s1, $0, 0x0000
        Inner_Loop:
                beq $s1, $t1, exit_in
                addi $s2, $s2, 0x0001
                addi $s1, $s1, 0x0001
                j Inner_Loop
        exit_in:
        addi $s0, $s0, 0x0001
        j Outer_Loop
exit:
addi $t2, $0, 0x0FA0
beq $s2, $t2, DONE
addiu $s0, $0, 0XDEAD
j SKIP
DONE:  addiu $s0, $0, 0XD08E
SKIP:  jr $ra
```